

Assignment 1 Report - Recipe App - Nicolas Grass

SDLC Model

For this assignment I used an Iterative and Incremental (Agile) SDLC model. Instead of attempting to design and complete every feature at the same time, I developed the application through successive increments. Each increment (or in my case commit) added another usable layer/feature to the project: the initial data/skeleton and database foundation, the recipe catalog, shopping list calculations, the web interface, and finally testing and documentation. This suited the project because its requirements became clearer as the data and domain logic were completed. For example, separating recipe and shopping responsibilities into modules made it possible to add and test each domain without first completing the whole interface, they both worked individually and later collectively.

The model was a practical fit for a small, single developer project. FastAPI, SQLite, and the server rendered Jinja2 templates allowed each increment to be integrated into one application without introducing separate deployment systems. The service modules also gave the catalog and shopping logic clear seams for focused tests. Decisions and changes were recorded in "ADR.md", and the AI usage log in "AI_USAGE.md" records AI assistance used during development.

SMART Goals

I used the following goals to make the project outcome specific and verifiable:

1. **Deliver a usable catalog.** By the project deadline, I want to provide a running web applications which displays recipes, allows the user to search for different recipes, and lets them view the specific recipe details. This goal was achieved through the catalog services, API and HTML pages.
2. **Support scaled and combined shopping requirements.** By the deadline allow users to select recipes and serving counts, scale ingredients by the amount of servings, combine matching ingredients into the same products, and estimate the whole package/product costs.
3. **Keep domain logic independent for understanding and testing.** Before completion separate catalog and shopping logic into their individual modules and ensure coverage for their business logic with automated tests. This was achieved with "app/catalog", "app/shopping" and the "tests/test_domain_services.py".
4. **Ensure the minimum business logic coverage for testing.** Measure the coverage for the two core services modules and surpass the minimum 70% threshold. After testing 17 out of 17 tests passed and reported a 99.03% combined coverage over both modules.

All of these goals follow the SMART goals framework, being specific to observe through the application behaviour, measurable through the actual features implemented and output of the application, achievable, relevant to both the actual app and assignment requirements and bounded by the deadline.

How I followed the model in practice

The project history shows the incremental progress. Firstly I established the project structure, skeleton and data sources ("supermarket_products.csv and recipes.json). Then I added the SQLite schema and seeding logic. The catalog module introduced the recipe queries, search and serving amount scaling. The shopping module added unit normalization, aggregation across ingredients from different recipes and packaged/product-based pricing. After these domains were implemented I then connected them to FastAPI endpoints and the Jinja2 interface. The core iterative and incremental commits show how the app was built step by step or feature by feature. The following commits refined the user experience, added new features such as the recipe images or fixed issues discovered while using the application. The final iterations focused on writing the tests, and completing documentation for the project ("README.md" and "AI_Usage.md").

Although I followed the Agile model, since I was working alone I assumed all roles within the development (Product Owner, Designer, Programmer and Tester). In addition the milestones and changes were decided informally. The base idea that I followed was attempting to implement one feature/iteration each day. Although at the start it was easy to accomplish, when implementing the UI I often found myself fixing one issue yet somehow another issue arising. This led to some design and testing decisions to be documented after implementation rather than reviewed before each iteration.

Looking back at the project I would change two things: Firstly I would start by writing the tests for the code. From the experience we are getting in class exercises I find it a lot easier to have the test and write code/functions that try and pass the tests, having the code first and writing the tests later was much more difficult than if I had implemented the tests first. Secondly I would have tried implementing the UI when I started writing the business logic and the two modules. I believe it would have helped me avoid certain bugs and issues that I found later in the code when trying to implement all of the features within the UI at once.

Running and Testing

The project setup and operational instructions are also provided in 'README.md'. The testing instructions are also located in the same file at the root of the project.

The tests target the catalog and shopping service modules rather than the framework routing. Their cases cover recipe search, serving size scaling, unit normalization, cross recipe ingredient aggregation, package/product selection and price estimates, and relevant empty or invalid inputs. Database dependent tests use a temporary SQLite database with mock values..

Important Files

"README.md" - installation, run instructions, API summary, and test command.

"ADR.md" - architecture decision records, including the SQLite schema and testing approach.

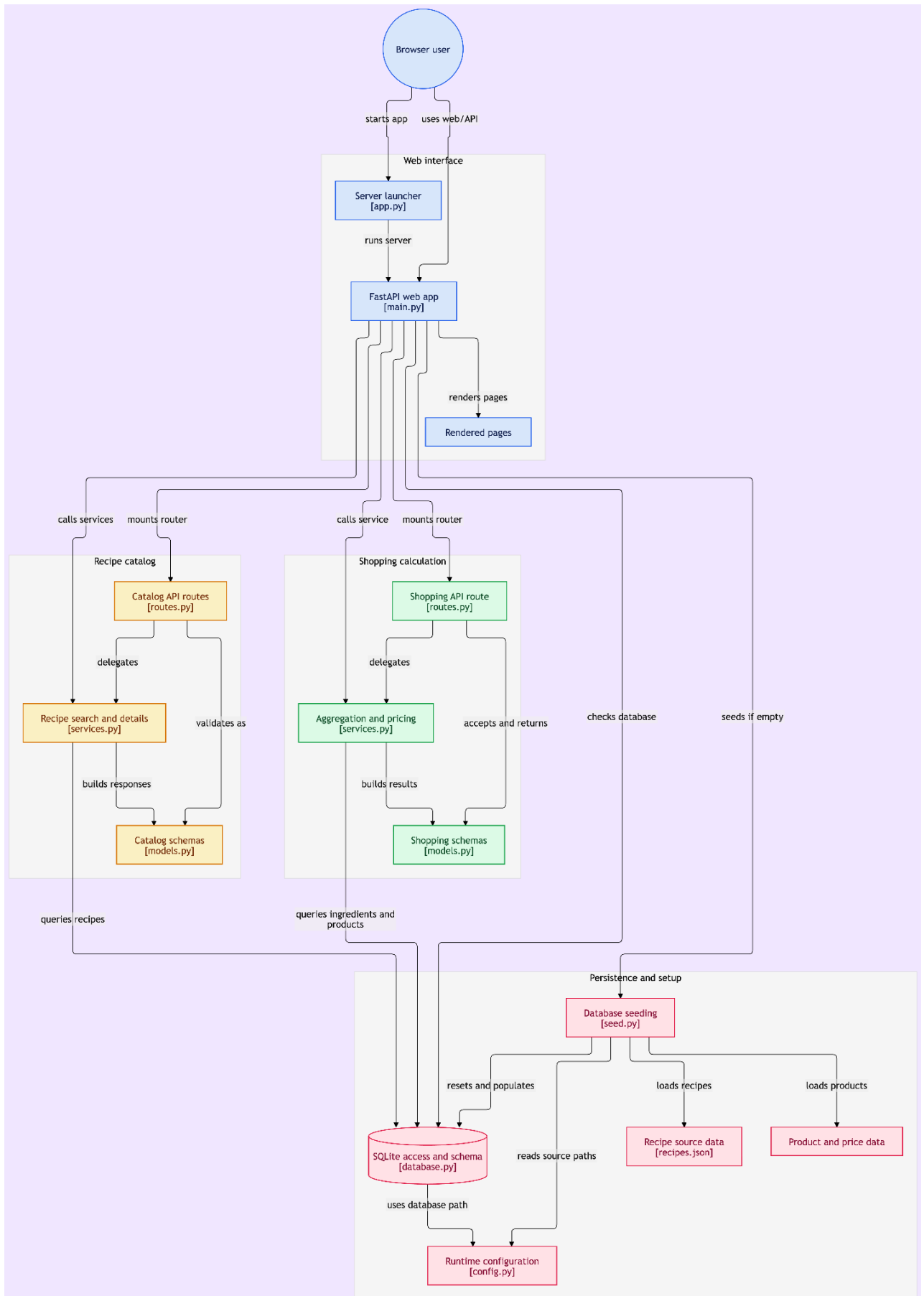
"AI_USAGE.md" - detailed record of AI prompts and how their output was used.

"app/database.py" - SQLite schema and connection setup.

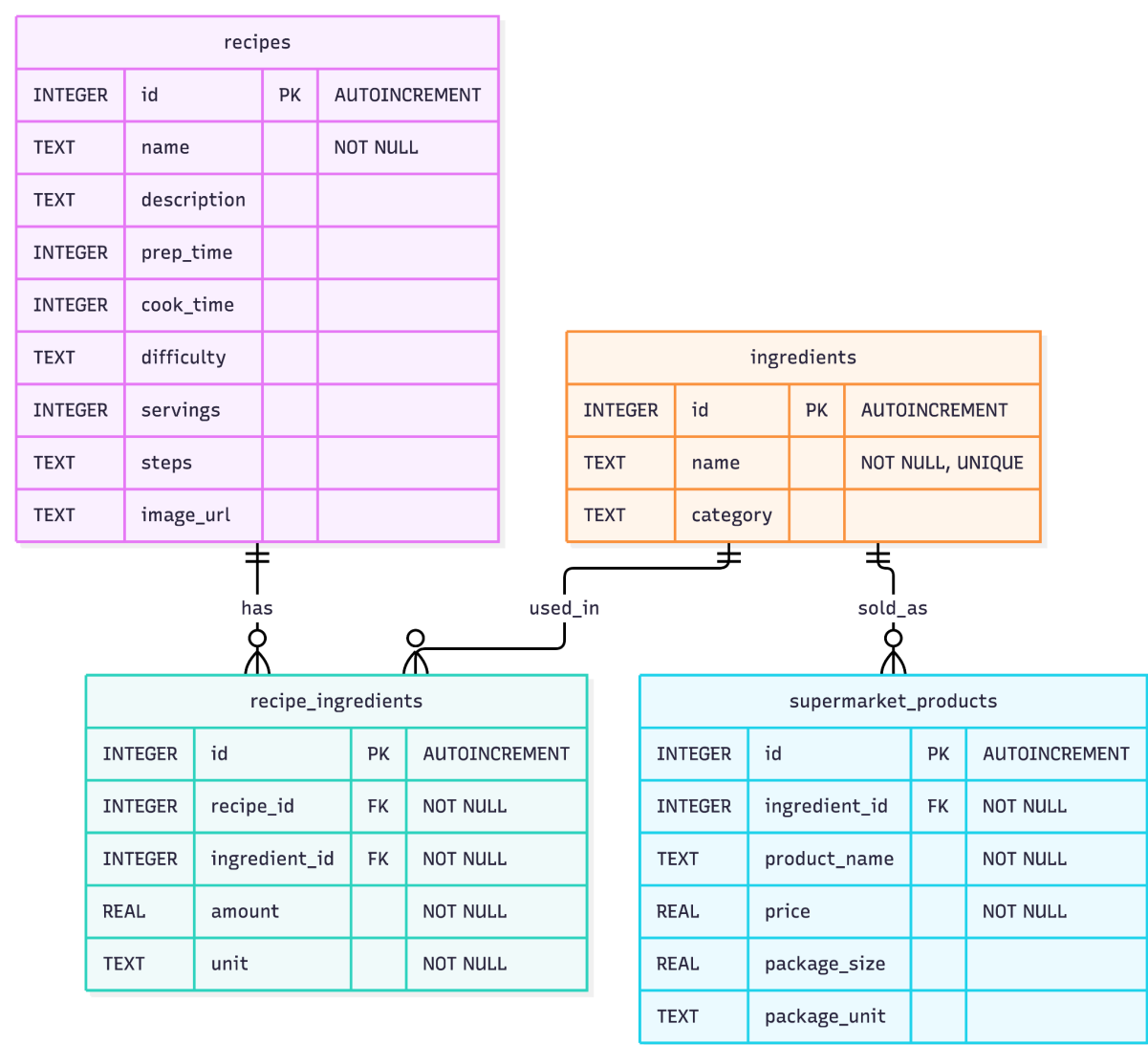
"app/catalog/" and **"app/shopping/"** - domain models, endpoints, and service logic.

"tests/test_domain_services.py" - automated domain service tests.

Architecture overview diagram



Database model/schema diagram



AI Disclosure Statement

I acknowledge the use of Google Gemini and ChatGPT to help plan and develop this project, generate and refine example data, assist with implementation and testing, troubleshoot issues, and improve project documentation. The prompts used include requests to plan the project, structure and milestones. To generate data for the products and recipes, write code, fix code and assist with writing documentation. The output of these prompts was used to improve project planning, data preparation, code, tests, and documentation. To view the detailed log please read AI_USAGE.md.